

Instructions:

- Ensure that this booklet contains 14 pages (counting each side).
- Each page of this booklet has a QR code and barcode at the bottom of the page. Do not write on or near these codes.
- Read through all the questions carefully.
- For any code which is provided to you, you should assume the proper `import` statements are there, even when not shown. That is, if a question asks you about the result of a function or if there is an error in the code it is not valid to say that there is an error because of missing `import` statements.
- Additionally, you do not have to write an import statement for functions from the `cmput274` module. You may assume that you implicitly have access to the `cmput274` module functions.
- Questions other than clarifying English non-course related words will not be answered. Proctors will only confirm or deny errors in the questions.
- All solutions must be placed in this booklet.
- If you need more space to complete an answer, use the extra pages at the back and clearly indicate that you have done so on the original answer page
- You may not use any concepts or functions we have not learned about yet in any of your answers. That means some of the things you are not allowed to use include, but are not limited to, loops, methods, Python `lists`, and variables defined outside a function. If you are concerned about if you can use something or not you should raise your hand and ask. A list of allowed functions is provided next for your consultation, which are always allowed unless explicitly forbidden by a question.
- Allowed functions from `cmput274` module: `LL`, `first`, `rest`, `empty`, `cons`, `isEmpty`, `foldr`, `foldl`, `map`, `filter`, and `buildList`
- Allowed built-in functions: `chr`, `ord`, `len`, `sqrt`, `ceil`, and `floor`
- Allowed keywords: `def`, `if`, `elif`, `else`, `lambda`, `and`, `or`, `not`, `True`, `False`, `return`
- Allowed Operators: all operators we have discussed are allowed, that includes arithmetic operators, sequence operators, and the ternary operator.
- Unless a question specifically asks you to you do **not** need to write function specifications for your functions.
- Unless otherwise specified by the question at hand you are always allowed to write helper functions where you find them necessary.
- Good luck :)

1. **Multiple Choice [5 marks]** — Unless otherwise specified by the individual question, circle **one** answer under each question.

(a) **[1 mark]** In your personal opinion is the statement “Functions are fun!” true or false?

- (A) True
- (B) False

(b) **[1 mark]** Consider the following function `foo`

```
def foo(x):  
    if x == 0:  
        return 5  
    if x < 0:  
        return x + 1  
    if x > 0:  
        return x - 1  
    return x//2
```

If we consider the input size of `foo` to be the term n , then which of the following is true?

- (A) $foo \in \Theta(n)$
- (B) $foo \in \Theta(n^2)$
- (C) $foo \in \Theta(1)$
- (D) $foo \in \Theta(\log n)$

(c) **[1 mark]** Is the statement “The domain of a function must be the same as its codomain” true or false?

- (A) True
- (B) False

(d) **[1 mark]** Is the statement “If a function is recursive then it is in $\Theta(n)$ ” true or false?

- (A) True
- (B) False

(e) **[1 mark]** Select the answer that most correctly finishes the following sentence: The purpose of the keyword `lambda` in Python is to

- (A) implement a higher-order function
- (B) write an anonymous function
- (C) define a fold function
- (D) evaluate an expression

2. Short Answer [12 marks]

- (a) [2 marks] Consider the following function definitions

$$f(x) \longrightarrow 3x^2 - 10$$

$$g(y) \longrightarrow \frac{y}{2} + 1$$

Write a similar function definition for the function p such that the function p always produces the same result as $f \circ g$

- (b) [2 marks] In your own words define what a *higher order function* is

- (c) [2 marks] Consider the Python function `f` below. Write an equivalent function `g` beside it that performs the exact same operation but without using the keywords `elif` or `else`.

```
def f(c1, c2, x):  
    if c1:  
        z = x*2  
    elif c2:  
        z = -1*x  
    else:  
        z = x  
    return z
```

```
def g(c1, c2, x):
```

(d) [2 marks] Write an expression that evaluates to the LList (29, 97, 38, 93) but you may not use the function LL. You may only use the functions `cons` and `empty`.

(e) [2 marks] In terms of the input size n determine the big-theta time complexity of the following function. Give an explanation of your choice in words, no formal proof is necessary.

```
def superSum(n):
    def summation(i):
        if i == 0:
            return 0
        return i+summation(i-1)
    if n == 0:
        return 0
    return summation(7) + superSum(n-1)
```

(f) [2 marks] In your own words describe what the `return` keyword of the Python language does.

3. **Evaluating Expressions [8 marks]** Each of the following are valid Python expressions, some using definitions from our `cmput274` module. For each of the following write what the result of evaluating the expression would be.

(a) `1 <= 5 and (2 == 3 or 10 > 15)`

(b) `10-3*5+1`

(c) `("Huge" + "north")[2:6]`

(d) `(lambda x : x*2+1)(3)`

(e) `first(rest(cons(1,
 cons(8,
 cons(12,
 cons(15, empty()))))))`

(f) `3*(2 if 1 > 5 else 3)`

(g) `map(lambda x: -x if x < 0 else x, (-2, 3, 5, -10, 0))`

(h) `foldr("cheer", lambda x, y: y + x, "")`

4. **Writing a Simple Python Function [5 marks]** The *Lambda Theater* has a specific pricing system for their movie tickets. The system is as follows:

- Regularly a movie ticket base price is 9.95
- On days of the month which are divisible by 5 but not 3 then the ticket base price is instead 7.65
- If the ticket purchaser is more than 59 years old then the ticket costs ten percent less than its base price for that day
- If the ticket purchaser is between no less than 12 years old and no more than 16 years old then the ticket costs twenty percent *more* than its base price for that day.

For example, if it is the 20th of October and the ticket is being purchased by a sixteen-year-old individual then the base price of the ticket would be 7.65, because 20 is divisible by 5 but not 3, and the individual would have to pay 20% more than that price because they are no less than 12 years old and no more than 16 years old.

Write the Python function `ticketPrice` that takes two parameters the first being the day of the month and the second being the age of the ticket purchaser. Your function should return the price that individual should pay for their ticket per the Lambda Theater pricing scheme described above.

You must also write the function specification for your function, including no less than three examples in the example section of your specification.

5. **Solving a Recursive Problem [5 marks]** For this question you must write the function `skipVals`. `skipVals` produces a `LList` that is the result of skipping over values of the given parameter `l` based on the following procedure

- (a) Begin by considering the first value of the given `LList` to be the “current value”.
- (b) Include the current value in your resultant `LList`
- (c) Skip over a number of values in the given `LList` equal to the current value, let the first value after all the skipped values be your new current value.
- (d) Repeat from the second step until there is no more work to do.

For example, given the result of `skipVals(LL(1, 2, 3, 0, 8, 7, 4))` is `LL(1, 3, 4)` because:

- The first current value is 1, so it is included in the result.
- As such, the 2 in the original `LList` is skipped. The next current value is 3.
- As 3 is the current value it is included in the result. Then three values are skipped over, so the 0, 8, and 7 are skipped over. The current value is then 4
- As 4 is the current value it is included in the result. The entire `LList` has been considered, and so there is no more work to do.

On the next page is the beginning of `skipVals` with an already provided function specification. Fill in the function and **complete your solution on the next page**.

Restrictions: You may not use the functions `foldr`, `foldl`, `map`, `filter`, or `buildList` in your solution. You may write any helper functions you need.

Hint: It may be useful to write a helper function that takes a natural number parameter `n` and a `LList` and returns the result of skipping `n` values of that `LList`.

```

def skipVals(l):
    '''
    skipVals returns the LList that is the result
        of skipping values in skipVals based
        on the sequence of values provided in
        the LList l itself

    l - LList of integers
    returns - LList of integers

    Examples
    skipVals(LL(1,2,3,0,8,7,4)) -> LL(1, 3, 4)
    skipVals(LL(1,1,1,5,1,1)) -> LL(1, 1, 1)
    skipVals(LL(10,1,3)) -> LL(10)
    '''

```

6. **Using Higher Order Functions [5 marks]** For this question you must solve the following Python function **without using explicit recursion**. That is, you cannot write a function that calls itself. Instead, you must use the higher-order functions provided in the `cmput274` module to solve these problems. As a reminder, the higher-order functions you have access to in the `cmput274` module are:

- `foldr(sequence, fn, base)`
- `foldl(sequence, fn, acc)`
- `map(fn, sequence)`
- `filter(pred, sequence)`
- `buildList(fn, nat)`

The order of parameters of each of these functions is given above, but it is expected you recall the behaviour of these functions from class.

Write the function `makeUpper` that takes a string parameter and returns a new string which is the result of making all lower case characters in the given parameter uppercase, and leaving all other characters unchanged.

For example:

```
makeUpper("abcDFG") -> "ABCDFG"  
makeUpper("99 Luftballons") -> "99 LUFTBALLONS"
```

Hint: The ASCII value for `'a'` is 97 and the lowercase letters continue sequentially until `'z'` which is 122. Similarly, the ASCII value for `'A'` is 65 and the uppercase letters continue sequentially until `'Z'` which is 90.

The built-in Python function `ord` takes a single character string and returns the natural number that is the ASCII value of that character. The reverse operation is performed by the function `chr` which takes a natural number and returns the single character string which is the character represented by that number in the ASCII table.

For example `ord('c')`->99 and `chr(102)`->'f'

Write your solution to this question on the following page.

Write your function `makeUpper` on this page.

7. **Writing Higher Order Functions [5 marks]** For this question you will be writing the higher-order function `composeMap`. This function has two `LList` parameters `fns` and `vals`. The parameter `fns` is a `LList` of unary functions that have a single integer parameter and return an integer. The parameter `vals` is a `LList` of integers.

The purpose of the function `composeMap` is to compose all the functions in the parameter `fns` together and return a new `LList` of integers that is the result of applying that function composition to each of the integers the `LList` `vals`.

For example:

```
composeMap(LL(lambda x : x*2, lambda x: x+1, lambda x:x*3),
           LL(1, 2, 3))
```

Would produce the `LList` (8, 14, 20). This is because we have composed the functions in our `fns` parameter together and applied those function compositions to each of the integers 1, 2, and 3 in our `vals` `LList`.

To simplify writing let us name the first function in our `LList` f , the second function in our `LList` g , and the third function in our `LList` h . Then applying $f \circ g \circ h$ to the first value of our `vals` `LList` would be the following:

$$\begin{aligned} f \circ g \circ h(1) \\ f \circ g(1 * 3) \\ f \circ g(3) \\ f(3 + 1) \\ f(4) \\ 4 * 2 \\ 8 \end{aligned}$$

The function specification for `composeMap` is provided on the following page. Write your solution to this problem on the following page.

```
def composeMap(fns, vals):
    '''
    composeMap composes the functions in fns together and maps
        that composition onto the LList vals.

    fns      - LList of (int->int)
    vals     - LList of ints
    returns  - LList of ints

    Examples:
        composeMap(LL(lambda x: x+1, lambda y: y*2),
                    LL(5, 8)) -> (11, 17)
        composeMap(LL(lambda x: x), LL(1, 2)) -> (1, 2)
    '''
```

This page is for rough work or to continue answering a question. **If you want this work to be graded** clearly indicate which question you are answering and where your answer starts and ends.